

TECHNICAL ARCHITECTURE SPECIFICATION

PGR Platform - Technical Architecture Specification

React + FastAPI + PostgreSQL

Postgraduate Research (PGR) Student Lifecycle Management Platform

Stack: React (frontend) + FastAPI (backend) + PostgreSQL (primary datastore)

Status: Baseline architecture, production grade, horizontally scalable

Audience: engineering, architecture, DevOps, QA, product

Contents

0. How to read this document

1. System overview

2. Architecture goals and principles

3. Logical architecture

3.1 Context view

3.2 Container responsibilities

4. Technology stack and rationale

5. Repository and file structure

6. Backend architecture

6.1 Layering

6.2 Request lifecycle and middleware

6.3 Configuration

6.4 Error handling

6.5 Dependency injection

7. Domain model and capability mapping

8. Data model (PostgreSQL)

8.1 Conventions

8.2 Core enumerations

8.3 Person and identity

8.4 Recruitment: opportunity and application

8.5 Admissions

8.6 Student record

8.7 Supervision

8.8 Progression

8.9 Funding

8.10 Thesis and examination

8.11 Completion and graduation

8.12 Workflow, task, notification, audit tables

8.13 Entity relationships (text ERD)

8.14 Indexing, partitioning, and performance

8.15 Migrations

9. Workflow, task, and notification engine

9.1 Model

9.2 Triggers

9.3 Timers and scheduled jobs

9.4 Guarantees

10. Integration architecture

10.1 Boundaries

10.2 Patterns

10.3 Adapter structure

11. API design

11.1 Conventions

11.2 Pagination, filtering, sorting

11.3 Standard envelopes

11.4 Idempotency and concurrency

11.5 Endpoint catalog (representative, per module)

11.6 OpenAPI

12. Authentication, authorization, RBAC, and audit

12.1 Authentication

12.2 Authorization model

12.3 Row scoping

12.4 Audit

13. Reporting and analytics

13.1 Read path separation

13.2 PGR Enterprise 360

13.3 Dashboards

13.4 Exports

14. Frontend architecture (React)

14.1 Contract

14.2 Recommended structure

14.3 Recommended libraries

15. Non functional requirements and service levels

16. Scalability and performance strategy

17. Security and compliance

18. Observability and operations

19. Deployment and infrastructure

19.1 Images and environments

19.2 Kubernetes shape

19.3 CI and CD

19.4 Data and backups

20. Testing strategy

21. MVP scope and phasing

22. Key design decisions

Appendix A. Machine readable data dictionary

Appendix B. Standard error codes

Appendix C. Environment variables

Appendix D. Glossary

0. How to read this document

1. System overview

2. Architecture goals and principles

3. Logical architecture

3.1 Context view

3.2 Container responsibilities

4. Technology stack and rationale

5. Repository and file structure

6. Backend architecture

6.1 Layering

6.2 Request lifecycle and middleware

6.3 Configuration

6.4 Error handling

6.5 Dependency injection

7. Domain model and capability mapping

8. Data model (PostgreSQL)

8.1 Conventions

8.2 Core enumerations

8.3 Person and identity

8.4 Recruitment: opportunity and application

8.5 Admissions

8.6 Student record

8.7 Supervision

8.8 Progression

8.9 Funding

8.10 Thesis and examination

8.11 Completion and graduation

8.12 Workflow, task, notification, audit tables

8.13 Entity relationships (text ERD)

8.14 Indexing, partitioning, and performance

8.15 Migrations

9. Workflow, task, and notification engine

9.1 Model

9.2 Triggers

9.3 Timers and scheduled jobs

9.4 Guarantees

10. Integration architecture

10.1 Boundaries

10.2 Patterns

10.3 Adapter structure

11. API design

11.1 Conventions

11.2 Pagination, filtering, sorting

11.3 Standard envelopes

11.4 Idempotency and concurrency

11.5 Endpoint catalog (representative, per module)

11.6 OpenAPI

12. Authentication, authorization, RBAC, and audit

12.1 Authentication

12.2 Authorization model

12.3 Row scoping

12.4 Audit

13. Reporting and analytics

13.1 Read path separation

13.2 PGR Enterprise 360

13.3 Dashboards

13.4 Exports

14. Frontend architecture (React)

14.1 Contract

14.2 Recommended structure

14.3 Recommended libraries

15. Non functional requirements and service levels

16. Scalability and performance strategy

17. Security and compliance

18. Observability and operations

19. Deployment and infrastructure

19.1 Images and environments

19.2 Kubernetes shape

19.3 CI and CD

19.4 Data and backups

20. Testing strategy

21. MVP scope and phasing

22. Key design decisions

Appendix A. Machine readable data dictionary

Appendix B. Standard error codes

Appendix C. Environment variables

Appendix D. Glossary

0. How to read this document

This file is the machine-oriented source of truth. It is dense and structured so that an engineer or an automated tool can parse entities, endpoints, and rules directly. A human-oriented PDF is produced from the same content with a cover page, contents, and reading layout.

Conventions used throughout:

- All identifiers are `snake_case` for database and Python, `camelCase` for JSON over the wire, `PascalCase` for React components and Python classes.
 - Primary keys are UUID version 4 unless stated.
 - Every persisted row carries `created_at`, `updated_at` timestamps in UTC.
 - Ranges are written with the word "to", for example "Jan 2026 to Dec 2026".
 - Arrows in diagrams use ASCII `->` and box characters, so the document stays copy safe.
-

1. System overview

The platform manages the full relationship between a person and the institution, from a research opportunity through recruitment, registration, progression, funding, thesis, examination, completion, and alumni status. It integrates with Research, Finance, and HR systems rather than replacing them.

The central object is not the student. It is the person, tracked across every identity the person can hold over time.

```
Person -> Research Opportunity -> Application -> Research Student ->  
Research Journey -> Funding -> Progression -> Thesis -> Outcome ->  
Alumni / Employee / Researcher
```

Funding is modelled as a cross cutting dimension, not a single stage, because a person can move between funding arrangements during one research journey.

The system owns the PGR lifecycle and the workflow around it. It does not own payroll, general ledger, research grant administration, or bulk document storage. Those remain in the systems of record and are reached through integration adapters.

2. Architecture goals and principles

Goals:

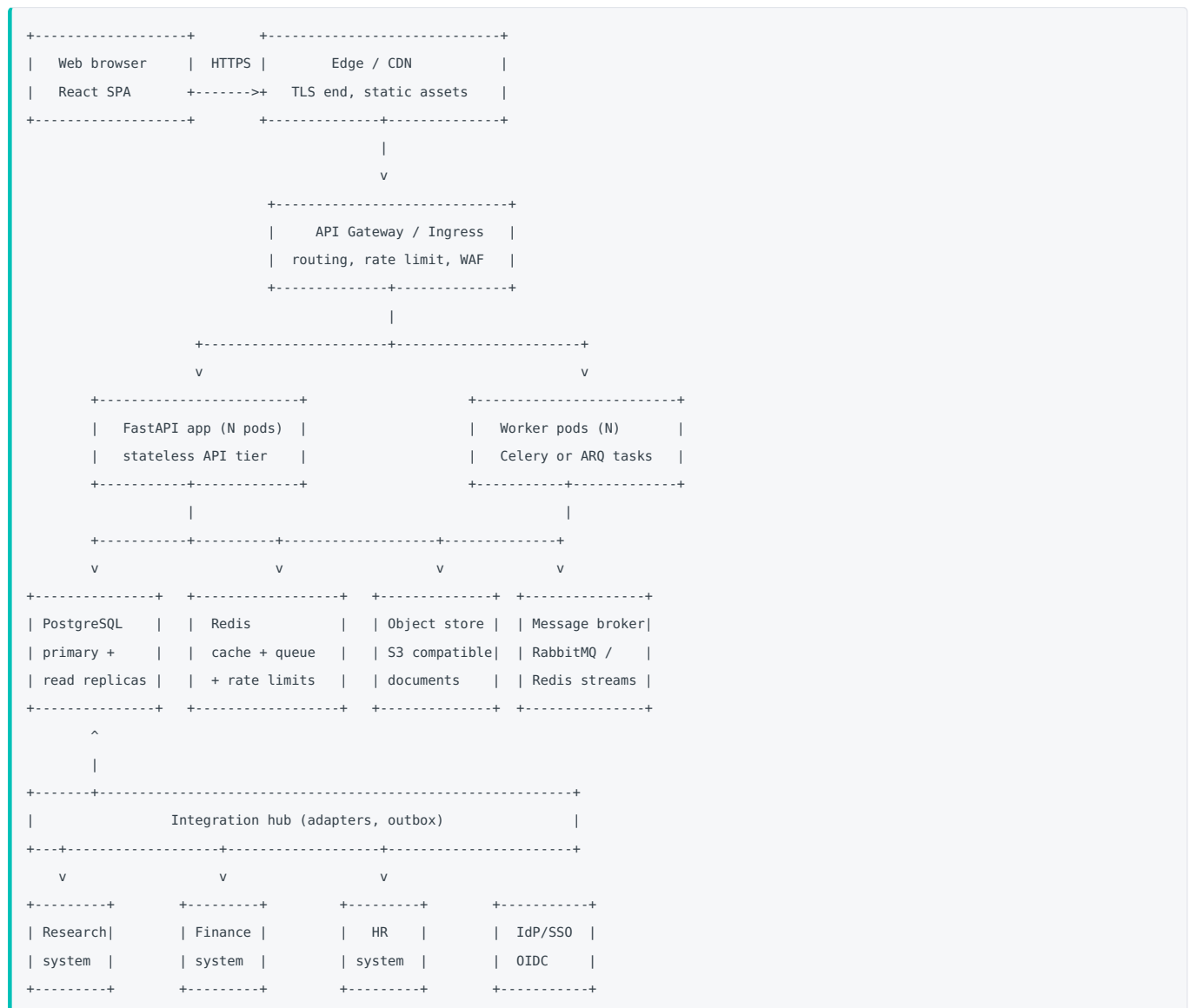
1. Model the person once and preserve one identity across applicant, student, employee, and alumni states.
2. Make the lifecycle configurable, so milestones, workflows, and assessment criteria can differ by institution and programme without code changes.
3. Keep external systems of record authoritative and integrate through clear boundaries.
4. Scale horizontally on the API and worker tiers, with the database tuned for read heavy reporting.
5. Produce a full and defensible audit trail for every academic and funding decision.

Principles:

- Modular monolith first. The backend is one deployable FastAPI application split into strict domain modules with private internals. Modules can be extracted into separate services later without rewriting business logic, because each module already talks to others through service interfaces and domain events, not shared tables.
 - Layered inside each module: API layer, service layer (business rules), repository layer (data access), domain models. No layer reaches around the one below it.
 - Explicit contracts. Pydantic schemas define every request and response. SQLAlchemy models never leak to the API layer.
 - Async by default for input and output bound work. CPU bound or long running work goes to background workers.
 - Configuration over hard coding. Lifecycle rules live in data, not in branches.
 - Fail closed on authorization. A request with no matching permission is denied.
-

3. Logical architecture

3.1 Context view



3.2 Container responsibilities

- React SPA: presentation, client routing, role aware views. Talks only to the API over HTTPS.
- API tier: stateless FastAPI pods. Handles synchronous requests, validation, authorization, and orchestration of services. Emits domain events to the outbox.
- Worker tier: consumes tasks and scheduled jobs. Runs notifications, milestone generation, funding expiry checks, integration sync, and report refresh.
- PostgreSQL: transactional store. Primary for writes, one or more read replicas for reporting and dashboards.
- Redis: response cache, rate limiter store, and short lived task broker for ARQ if chosen.
- Object store: thesis files and evidence documents by reference. Large document management can also stay external.
- Message broker: reliable delivery of domain events to the integration hub and between modules if they are later split.

4. Technology stack and rationale

Layer	Choice	Reason
Frontend	React 18, TypeScript, Vite	Component model fits role based dashboards and journey timelines. Provided by the frontend team.
State and data fetching	TanStack Query, Zustand	Server cache and light client state without heavy boilerplate.
Backend framework	FastAPI on Python 3.12	Async, typed, automatic OpenAPI, strong Pydantic validation.
ORM	SQLAlchemy 2.0 async	Mature, supports async sessions and precise SQL when needed.
Migrations	Alembic	Standard for SQLAlchemy, reviewable migration scripts.
Validation and settings	Pydantic v2, pydantic-settings	One validation model across API and configuration.
Auth	OAuth2 with JWT access and refresh, OIDC for SSO	Works with institutional identity providers.
Background work	Celery with Redis or RabbitMQ, or ARQ for lighter needs	Scheduled and event driven jobs off the request path.
Database	PostgreSQL 16	Strong relational integrity, JSONB for flexible config, partitioning, materialized views.
Cache and broker	Redis 7	Cache, rate limits, and queue.
Object storage	S3 compatible (AWS S3 or MinIO)	Document references and evidence.
Containerisation	Docker	Reproducible builds.
Orchestration	Kubernetes	Horizontal scaling, rolling deploys, health probes.
Observability	OpenTelemetry, Prometheus, Grafana, Sentry	Traces, metrics, error tracking.
CI and CD	GitHub Actions or GitLab CI	Test, build, scan, deploy.

Rationale note: the modular monolith with async FastAPI gives a small team production capability without the operational cost of many services on day one. The module boundaries and event contracts keep the door open to service extraction when a specific module needs independent scaling.

5. Repository and file structure

Monorepo. One repository holds backend, frontend, infrastructure, and documentation, so changes that cross the API contract move together.

```
pgr-platform/
  backend/
    app/
      main.py          # FastAPI app factory, router registration
    core/
      config.py        # Settings from environment
      database.py      # Async engine, session factory
      security.py      # Password hashing, JWT, token helpers
      dependencies.py  # Shared FastAPI dependencies
      authorization.py # Permission checks, row scoping
      pagination.py    # Cursor and offset helpers
      errors.py        # Exception types and handlers
      events.py        # Domain event base, outbox writer
      middleware.py    # Request id, logging, timing
      logging.py       # Structured logging setup
      telemetry.py     # OpenTelemetry init
    modules/
      person/
        router.py      # HTTP endpoints
        schemas.py     # Pydantic request and response models
        models.py      # SQLAlchemy ORM models
        service.py     # Business rules, orchestration
        repository.py  # Data access, queries
        events.py      # Events this module emits or handles
        constants.py   # Enums, status values
      identity/        # Users, roles, permissions, auth flows
      recruitment/     # Opportunities, applications, pipeline
      admissions/      # Offers, acceptance, onboarding
      student_record/  # Core PGR student profile
      supervision/    # Supervisors and supervisory teams
      progression/     # Milestones, reviews, decisions
      funding/         # Funding arrangements, stipend link
      thesis/          # Submission, examination, corrections
      completion/      # Completion, award, graduation, alumni
      workflow/        # Workflow, task, notification engine
      integration/     # Adapters, outbox dispatch, webhooks
      reporting/       # Read models, dashboards, exports
    api/
      v1/
        routes.py      # Aggregates module routers under /api/v1
    db/
      base.py          # Declarative base, naming convention
      session.py       # get_session dependency
      migrations/      # Alembic environment and versions
    workers/
      app.py           # Celery or ARQ app
      schedules.py     # Periodic job definitions
      tasks/
        notifications.py
        milestones.py
        funding.py
        integration_sync.py
        reporting_refresh.py
    tests/
      unit/
      integration/
```

```
e2e/
  conftest.py
alembic.ini
pyproject.toml          # Dependencies, tool config
Dockerfile
.env.example
frontend/               # Provided by the frontend team, see section 14
infra/
  docker/
    docker-compose.yml  # Local full stack
k8s/
  base/                 # Deployments, services, config maps
  overlays/
    dev/
    staging/
    prod/
  terraform/           # Cloud infrastructure as code
docs/
  architecture/
  adr/                 # Architecture decision records
  api/                 # Generated OpenAPI, guides
.github/
  workflows/           # CI pipelines
README.md
```

Module anatomy is uniform. Every module has the same seven files, so a new engineer can navigate any capability the same way. The only public surface of a module is its `router.py` and its `service.py` interface. Other modules call the service, never the repository or models of a peer module.

6. Backend architecture

6.1 Layering

```
HTTP request
-> Router (validation via Pydantic, authz dependency)
    -> Service (business rules, transactions, events)
        -> Repository (SQLAlchemy queries)
            -> Database
    <- Domain object mapped to response schema
<- JSON response
```

Rules:

- Routers contain no business logic. They validate input, resolve the current user, call a service method, and shape the response.
- Services own transactions. A service method opens a unit of work, calls one or more repositories, writes any domain events to the outbox in the same transaction, then commits.
- Repositories contain queries only. They return domain objects or plain rows, never HTTP concerns.
- Cross module calls go service to service. If `admissions` needs a person, it calls `person_service.get_or_create`, not the person tables.

6.2 Request lifecycle and middleware

Order of processing:

1. TLS ends at the ingress. The API sees plain HTTP inside the cluster.
2. Middleware attaches a request id, starts a trace span, and records start time.
3. Authentication dependency validates the JWT and loads the principal.
4. Authorization dependency checks the required permission and applies row scoping.
5. The route handler runs, calls the service, and returns a Pydantic model.
6. Exception handlers convert known errors to the standard error envelope.
7. Middleware logs method, path, status, duration, and request id as one structured line.

6.3 Configuration

All configuration comes from environment variables, loaded once into a typed settings object. No secrets in code. Local development uses `.env`. Clusters use secrets and config maps. See Appendix C for the variable list.

6.4 Error handling

One exception hierarchy in `core/errors.py`:

- `AppError` base with `code`, `message`, `http_status`.
- `NotFoundError`, `ConflictError`, `ValidationAppError`, `AuthError`, `PermissionError`, `IntegrationError`, `WorkflowError`.

A single handler renders the standard envelope in Appendix B. Unhandled exceptions return a generic 500 with a request id and never leak internals.

6.5 Dependency injection

FastAPI dependencies provide the database session, the current principal, and the permission guard. Services receive their repositories through constructor injection in tests and through a small factory in the app, so units can be tested without a database.

7. Domain model and capability mapping

Twelve capabilities map to backend modules and to database schemas.

#	Capability	Module	Owns (primary tables)
1	Person and identity	person, identity	person, person_relationship, users, roles
2	Research demand	recruitment	research_opportunity
3	PGR recruitment	recruitment	application, candidate_stage_history
4	Admission and onboarding	admissions	offer, onboarding_task
5	Research student record	student_record	student, programme, research_project
6	Research supervision	supervision	supervisor_relationship
7	Research progression	progression	milestone_definition, milestone, progression_review
8	Funding and stipend	funding	funding_arrangement, funding_source, research_award
9	Thesis and examination	thesis	thesis, examination, examiner_nomination
10	Completion and graduation	completion	completion, award
11	Integration and reporting	integration, reporting	outbox_event, integration_log, report views
12	Lifecycle analytics	reporting	materialized views, dashboard read models

Cross cutting engines: workflow, task, notification, and audit are shared services used by every module.

8. Data model (PostgreSQL)

8.1 Conventions

- UUID primary keys named `id`.
- Foreign keys named `<entity>_id`.
- Status columns use PostgreSQL enum types or a constrained text with a check.
- Timestamps `created_at`, `updated_at` default to `now()` at UTC. `updated_at` maintained by a trigger.
- Soft delete only where history matters, using `deleted_at`. Most lifecycle records are never hard deleted.
- JSONB used for configurable, sparse, or institution specific attributes, with a documented shape.
- Money is stored as `numeric(14,2)` with a separate `currency` `char(3)`. The platform records funding relationships, not payments.

8.2 Core enumerations

```
person_relationship_type: applicant, student, employee, alumni, researcher
opportunity_status: draft, approved, open, recruiting, filled, closed
application_route: opportunity_led, student_led
candidate_stage: prospect, applicant, under_assessment, shortlisted,
                 interview, selected, offer_made, offer_accepted,
                 rejected, withdrawn, converted
offer_status: draft, issued, accepted, declined, expired, withdrawn
student_status: prospective, registered, active, on_leave, suspended,
               completed, withdrawn, terminated
study_mode: full_time, part_time
supervisor_role: primary, co_supervisor, additional
supervision_status: assigned, accepted, active, changed, ended
milestone_status: not_started, due, submitted, under_review, decided, overdue
progression_outcome: progress, progress_with_conditions, further_review,
                   transfer_award, withdraw, terminate
funding_type: research_council, university_scholarship, external, self_funded
funding_status: planned, active, changed, ended
thesis_status: preparation, intention_to_submit, submitted, under_examination,
              corrections, resubmission, approved, failed
examination_outcome: pass, pass_with_corrections, major_corrections,
                   resubmission, fail
completion_status: pending, requirements_met, award_confirmed, graduated
workflow_state: created, active, waiting, completed, cancelled
task_status: open, in_progress, blocked, done, cancelled
notification_status: queued, sent, read, failed
```

8.3 Person and identity

Table `person`

Column	Type	Notes
<code>id</code>	uuid pk	
<code>external_person_ref</code>	text unique null	link to IdP or HR person if matched
<code>given_name</code>	text not null	
<code>family_name</code>	text not null	
<code>preferred_name</code>	text null	
<code>date_of_birth</code>	date null	
<code>email</code>	citext unique	primary contact
<code>nationality</code>	text null	
<code>created_at, updated_at</code>	timestamp tz	

Table `person_relationship` gives the person its many identities over time.

Column	Type	Notes
id	uuid pk	
person_id	uuid fk -> person.id	
relationship_type	enum person_relationship_type	
valid_from	date not null	
valid_to	date null	null means current
source_system	text null	which system asserted it

Index: (person_id, relationship_type), partial index where valid_to is null.

Table users (identity module) links a login to a person.

Column	Type	Notes
id	uuid pk	
person_id	uuid fk null	staff logins may not be a PGR person
email	citext unique not null	
password_hash	text null	null when SSO only
is_active	boolean default true	
last_login_at	timestamp null	

RBAC tables: roles, permissions, role_permission, user_role. See section 12.

8.4 Recruitment: opportunity and application

Table research_opportunity

Column	Type	Notes
id	uuid pk	
title	text not null	
research_area_id	uuid fk -> research_area.id	
department_id	uuid fk -> department.id	
principal_supervisor_id	uuid fk -> person.id null	
research_award_id	uuid fk -> research_award.id null	funding source context
funding_source_id	uuid fk -> funding_source.id null	
stipend_amount	numeric(14,2) null	
currency	char(3) null	
eligibility	text null	
start_date	date null	
expected_duration_months	int null	
positions_available	int default 1	
status	enum opportunity_status default draft	

State transitions enforced in service: draft -> approved -> open -> recruiting -> filled -> closed.

Table application

Column	Type	Notes
id	uuid pk	
person_id	uuid fk -> person.id	applicant identity
route	enum application_route	opportunity_led or student_led
research_opportunity_id	uuid fk null	null when student led
research_area_id	uuid fk null	for student led route
proposal_document_ref	text null	object store reference
current_stage	enum candidate_stage default applicant	
submitted_at	timestampz null	

Table `candidate_stage_history` records every pipeline move for audit and analytics.

Column	Type	Notes
id	uuid pk	
application_id	uuid fk	
from_stage	enum candidate_stage null	
to_stage	enum candidate_stage not null	
moved_by_user_id	uuid fk	
reason	text null	
moved_at	timestampz default now	

Table `application_assessment`

Column	Type	Notes
id	uuid pk	
application_id	uuid fk	
assessor_user_id	uuid fk	
criteria	jsonb	configurable criteria and scores
decision	text	recommended, conditionally_recommended, rejected, request_info
rationale	text	defensible trail
assessed_at	timestampz default now	

8.5 Admissions

Table `offer`

Column	Type	Notes
id	uuid pk	
application_id	uuid fk	
status	enum offer_status default draft	
conditions	jsonb null	
issued_at, responded_at, expires_at	timestampz null	

Table `onboarding_task` is created on offer acceptance and is a view over the shared task engine, filtered by context.

8.6 Student record

Table `student`

Column	Type	Notes
id	uuid pk	
person_id	uuid fk -> person.id	one identity preserved from applicant
student_ref	text unique	human readable id
programme_id	uuid fk -> programme.id	
department_id	uuid fk	
research_area_id	uuid fk	
start_date	date	
expected_end_date	date	
study_mode	enum study_mode	
status	enum student_status default registered	

Key rule: when an applicant converts to a student, the same `person_id` is reused. No new person is created.

Table `programme`, `department`, `school`, `research_area` are reference tables with `id`, `name`, `code`, and parent links where relevant.

Table `research_project`

Column	Type	Notes
id	uuid pk	
student_id	uuid fk	
research_topic	text	
research_award_id	uuid fk null	
research_group	text null	

8.7 Supervision

Table `supervisor_relationship`

Column	Type	Notes
id	uuid pk	
student_id	uuid fk	
supervisor_person_id	uuid fk -> person.id	
role	enum supervisor_role	
status	enum supervision_status default assigned	
valid_from	date	
valid_to	date null	null means current

History is preserved. A supervisor change ends one row and opens another rather than editing in place. Index `(student_id)` and partial index where `valid_to` is null.

8.8 Progression

Table `milestone_definition` makes progression configurable per programme.

Column	Type	Notes
id	uuid pk	
programme_id	uuid fk	
name	text	for example Confirmation Review
trigger	jsonb	for example months after registration
due_offset_days	int	
required_documents	jsonb	
review_panel	jsonb	roles or named panel
assessment_criteria	jsonb	
possible_outcomes	jsonb	subset of progression_outcome

Table `milestone` is a generated instance for a student.

Column	Type	Notes
id	uuid pk	
student_id	uuid fk	
milestone_definition_id	uuid fk	
due_date	date	
status	enum milestone_status default not_started	

Table `progression_review`

Column	Type	Notes
id	uuid pk	
milestone_id	uuid fk	
student_submission_ref	text null	
supervisor_assessment	jsonb null	
panel_decision	enum progression_outcome null	
decided_by_user_id	uuid fk null	
decided_at	timestampz null	
rationale	text null	

On a decision, the service records the outcome, updates the milestone, and asks the workflow engine to generate the next milestone.

8.9 Funding

Funding is separate from student status so a student can hold several arrangements over time.

Table `funding_source`, `research_award` are reference and award records with funder, award reference, period, and total value.

Table `funding_arrangement`

Column	Type	Notes
id	uuid pk	
student_id	uuid fk	
funding_type	enum funding_type	
funding_source_id	uuid fk null	
research_award_id	uuid fk null	
stipend_amount	numeric(14,2) null	
currency	char(3) null	
valid_from	date	
valid_to	date null	
status	enum funding_status default planned	

A funding change closes the current arrangement, opens the new one, writes an audit event, and emits an integration event so Finance is notified. Payment stays in Finance.

8.10 Thesis and examination

Table `thesis`

Column	Type	Notes
id	uuid pk	
student_id	uuid fk	
title	text null	
status	enum thesis_status default preparation	
intention_to_submit_at	timestampz null	
submitted_at	timestampz null	
document_ref	text null	object store or external repository

Table `examiner_nomination`

Column	Type	Notes
id	uuid pk	
thesis_id	uuid fk	
examiner_person_id	uuid fk	
examiner_type	text	internal, external
approved	boolean default false	
approved_by_user_id	uuid fk null	

Table `examination`

Column	Type	Notes
id	uuid pk	
thesis_id	uuid fk	
viva_date	date null	
outcome	enum examination_outcome null	
decided_at	timestampz null	

8.11 Completion and graduation

Table `completion`

Column	Type	Notes
id	uuid pk	
student_id	uuid fk	
status	enum completion_status default pending	
requirements_met_at	timestamptz null	
award_confirmed_at	timestamptz null	
graduation_date	date null	

On graduation, the service closes funding, sets student status to completed, records the award, and opens an alumni relationship on the same person. The person identity continues.

8.12 Workflow, task, notification, audit tables

Table `workflow_definition`, `workflow_instance`, `task`, `notification`, `audit_event`. See section 9 and section 12 for structure and behaviour.

Table `outbox_event` for reliable integration.

Column	Type	Notes
id	uuid pk	
aggregate_type	text	for example funding_arrangement
aggregate_id	uuid	
event_type	text	for example funding.changed
payload	jsonb	
created_at	timestamptz default now	
dispatched_at	timestamptz null	null until delivered

8.13 Entity relationships (text ERD)

```

person 1---* person_relationship
person 1---1 users (optional)
person 1---* application          (as applicant)
person 1---1 student              (as student, via person_id)
research_opportunity 1---* application
application 1---* candidate_stage_history
application 1---* application_assessment
application 1---0..1 offer
student 1---* research_project
student 1---* supervisor_relationship
student 1---* milestone
milestone 1---1 progression_review
student 1---* funding_arrangement
student 1---1 thesis
thesis 1---* examiner_nomination
thesis 1---1 examination
student 1---1 completion
programme 1---* milestone_definition
research_award 1---* funding_arrangement

```

8.14 Indexing, partitioning, and performance

- Index every foreign key used in joins and every status column used in dashboard filters.
- Composite indexes for common queries, for example `funding_arrangement (student_id, status)` and `milestone (status, due_date)`.
- Partition high volume append tables by month: `audit_event`, `candidate_stage_history`, `notification`, `outbox_event`.
- Use partial indexes for current records, for example supervisor and funding rows where `valid_to` is null.
- Reporting reads run against read replicas and materialized views, not the write primary.

8.15 Migrations

Alembic manages schema. Rules:

- One migration per change set, reviewed like code.
 - Migrations are forward only in production, with a tested down path kept for staging.
 - Data backfills that touch large tables run in batches inside a worker, not inside the schema migration.
 - Enum changes add values in a migration and never rename in place without a two step deploy.
-

9. Workflow, task, and notification engine

Almost every lifecycle event creates work for a person. The platform ships one reusable engine rather than hard coding each flow.

9.1 Model

```
workflow_definition      # a named state machine, versioned
  states: [ ... ]
  transitions: [ {from, to, on_event, guard, actions} ]

workflow_instance        # a running flow bound to an aggregate
  definition_id, aggregate_type, aggregate_id, current_state, context jsonb

task                    # a unit of human work
  workflow_instance_id, assignee_role or assignee_user_id,
  title, due_at, status, payload jsonb

notification            # an outbound message
  recipient_user_id, channel, template, payload, status
```

9.2 Triggers

Events raise transitions. Examples:

- `offer.accepted` starts the onboarding workflow and creates onboarding tasks.
- `milestone.due` notifies student and supervisor and opens a review task.
- `thesis.submitted` creates the examiner nomination workflow.
- `funding.expiring_90d` notifies the administrator to plan renewal.
- `progression.failed` opens the decision workflow.

9.3 Timers and scheduled jobs

A scheduled worker runs periodic checks:

- generate milestones that fall due from `milestone_definition`.
- flag funding arrangements expiring within 90 days.
- escalate tasks past their due date.
- refresh reporting materialized views.

9.4 Guarantees

Task creation and the triggering state change happen in one database transaction. Notifications are queued through the outbox so a broker outage never loses a message. Delivery is at least once, and templates are idempotent by notification id.

10. Integration architecture

The platform integrates with Research, Finance, HR, an identity provider, and document or repository systems. It never becomes any of them.

10.1 Boundaries

External system	Platform reads	Platform writes	Direction
Research	awards, projects, funding context	nothing	inbound to platform
Finance	payment status by reference	funding relationship events	both
HR	employee identity, contract facts	nothing	inbound
Identity provider	authentication, group claims	nothing	inbound
Document repository	document metadata	document references	both

10.2 Patterns

- Anti corruption layer. Each adapter translates external shapes into platform domain terms, so external changes do not ripple into the core.
- Outbox and dispatcher. Domain events are written to `outbox_event` inside the same transaction as the state change. A dispatcher worker publishes them to the broker and marks them dispatched. This gives reliable, ordered, at least once delivery.
- Inbound webhooks and batch. External systems push updates to signed webhook endpoints, or the platform pulls on a schedule for systems without webhooks.
- Idempotency. Every inbound message carries a source id. The platform stores processed ids and ignores repeats.

10.3 Adapter structure

```
integration/
  adapters/
    research_adapter.py    # map award and project context in
    finance_adapter.py     # send funding events, read payment status
    hr_adapter.py          # map employee identity in
    idp_adapter.py         # OIDC token validation, group mapping
    document_adapter.py    # store and fetch references
  dispatcher.py           # reads outbox, publishes, marks dispatched
  webhooks.py             # signed inbound endpoints
  integration_log table    # every inbound and outbound call recorded
```

11. API design

11.1 Conventions

- Base path `/api/v1`. The version is in the URL. Breaking changes create `/api/v2`.
- Resource oriented URLs, plural nouns, for example `/api/v1/students`.
- JSON request and response, `camelCase` fields.
- Standard verbs: GET list and read, POST create, PATCH partial update, PUT full replace where used, DELETE where a real delete is allowed.
- State changes that are not simple field edits use action sub resources, for example `POST /api/v1/applications/{id}/advance` with a target stage in the body. This keeps transitions explicit and auditable.

11.2 Pagination, filtering, sorting

- List endpoints accept `limit` (default 25, max 200) and either `offset` or an opaque `cursor`. Cursor pagination is used on large, append heavy collections.
- Filtering uses explicit query parameters per field, for example `status=active`. No arbitrary query language over the wire.
- Sorting uses `sort=field` and `order=asc|desc` with an allow list per resource.

11.3 Standard envelopes

List response:

```
{
  "data": [ { "id": "..." } ],
  "page": { "limit": 25, "nextCursor": "opaque-or-null", "total": 128 }
}
```

Error response, see Appendix B for codes:

```
{
  "error": {
    "code": "not_found",
    "message": "Student not found",
    "requestId": "req_01H...",
    "details": []
  }
}
```

11.4 Idempotency and concurrency

- Unsafe POST actions accept an `Idempotency-Key` header. The platform stores the key with the result for 24 hours and returns the same result on a repeat.
- Optimistic concurrency uses an `ETag` from a row version. A `PATCH` with a stale `If-Match` returns 409.

11.5 Endpoint catalog (representative, per module)

Identity and auth

POST	<code>/api/v1/auth/login</code>	password grant, returns access + refresh
POST	<code>/api/v1/auth/refresh</code>	exchange refresh for new access
POST	<code>/api/v1/auth/logout</code>	revoke refresh
GET	<code>/api/v1/auth/sso/callback</code>	OIDC redirect handling
GET	<code>/api/v1/me</code>	current principal, roles, permissions

Person

GET	/api/v1/persons	
POST	/api/v1/persons	
GET	/api/v1/persons/{id}	
PATCH	/api/v1/persons/{id}	
GET	/api/v1/persons/{id}/relationships	applicant, student, employee, alumni
GET	/api/v1/persons/{id}/timeline	full lifecycle across identities

Recruitment

GET	/api/v1/opportunities	
POST	/api/v1/opportunities	
PATCH	/api/v1/opportunities/{id}	
POST	/api/v1/opportunities/{id}/transition	{toStatus}
GET	/api/v1/applications	
POST	/api/v1/applications	opportunity_led or student_led
GET	/api/v1/applications/{id}	
POST	/api/v1/applications/{id}/advance	{toStage, reason}
POST	/api/v1/applications/{id}/assess	{criteria, decision, rationale}
GET	/api/v1/recruitment/pipeline	counts by stage for a filter

Admissions

POST	/api/v1/applications/{id}/offer	create offer
POST	/api/v1/offers/{id}/issue	
POST	/api/v1/offers/{id}/accept	triggers student creation
POST	/api/v1/offers/{id}/decline	

Student record

GET	/api/v1/students	
GET	/api/v1/students/{id}	
PATCH	/api/v1/students/{id}	
GET	/api/v1/students/{id}/project	
GET	/api/v1/students/{id}/summary	journey, funding, supervisors

Supervision

GET	/api/v1/students/{id}/supervisors	
POST	/api/v1/students/{id}/supervisors	assign
POST	/api/v1/supervisors/{relId}/end	end relationship, preserve history
GET	/api/v1/supervisors/{personId}/students	supervisor caseload

Progression

GET	/api/v1/programmes/{id}/milestone-definitions	
POST	/api/v1/programmes/{id}/milestone-definitions	
GET	/api/v1/students/{id}/milestones	
POST	/api/v1/milestones/{id}/submit	
POST	/api/v1/milestones/{id}/decide	{outcome, rationale}

Funding

GET	/api/v1/students/{id}/funding	
POST	/api/v1/students/{id}/funding	create arrangement
POST	/api/v1/funding/{id}/change	close current, open new
POST	/api/v1/funding/{id}/end	

Thesis and examination

```
POST /api/v1/students/{id}/thesis/intention
POST /api/v1/theses/{id}/submit
POST /api/v1/theses/{id}/examiners          nominate
POST /api/v1/examiner-nominations/{id}/approve
POST /api/v1/theses/{id}/examination/outcome {outcome}
```

Completion

```
POST /api/v1/students/{id}/completion/confirm
POST /api/v1/students/{id}/graduation      sets alumni relationship
```

Workflow, tasks, notifications

```
GET /api/v1/tasks                          my queue, filters by role
POST /api/v1/tasks/{id}/complete
GET /api/v1/notifications
POST /api/v1/notifications/{id}/read
```

Reporting

```
GET /api/v1/dashboards/executive
GET /api/v1/dashboards/supervisor
GET /api/v1/dashboards/administrator
GET /api/v1/reports/pgr-enterprise-360    multi lens read model
POST /api/v1/exports                      async export job, returns job id
GET /api/v1/exports/{jobId}
```

11.6 OpenAPI

FastAPI generates OpenAPI 3.1 automatically. The schema is published at `/api/v1/openapi.json` and a documentation site is generated in CI so the frontend team and integrators work from a single contract.

12. Authentication, authorization, RBAC, and audit

12.1 Authentication

- Primary path is OIDC against the institutional identity provider. The platform validates the token, maps identity provider groups to platform roles, and issues its own short lived access token plus a refresh token.
- A local password path exists for service accounts and non SSO users. Passwords are hashed with a strong algorithm. Access tokens are short lived. Refresh tokens are stored hashed and can be revoked.

12.2 Authorization model

Role based access with fine grained permissions and row scoping.

```
permission    # verb on a resource, for example student.read, funding.change
role          # a named set of permissions
role_permission
user_role
```

Roles from the blueprint, each with a default permission set:

- Student: own journey read, own submissions write.
- Supervisor: read and act on own supervisees only.
- PGR Administrator: manage lifecycle across the assigned unit.
- Academic or Panel: assess progression.
- Admissions or Recruitment: manage applications.
- Research Office: read research and funding relationships.
- Finance: read funding and stipend relationships.
- HR: read relevant person and workforce relationship.
- Registry: manage registration, completion, and statutory data.
- Institution Administrator: configure programmes, workflows, and rules.
- Executive: read only enterprise analytics.

12.3 Row scoping

A permission grants a verb. Scoping decides which rows. A supervisor with `student.read` sees only students linked to an active `supervisor_relationship` for that supervisor. Scoping runs as a query filter in the repository, applied from the principal, so it cannot be bypassed by a crafted request.

12.4 Audit

Every state change writes an `audit_event` with actor, action, aggregate, before and after summary, timestamp, and request id. Academic and funding decisions carry a rationale. Audit rows are append only and partitioned by month. Reads of sensitive data can also be audited where policy requires.

13. Reporting and analytics

13.1 Read path separation

Dashboards and reports never query the write primary. They read from replicas and from materialized views that pre join lifecycle data.

13.2 PGR Enterprise 360

One population, several lenses, built as materialized views refreshed on a schedule and on key events:

- Student lens: who the PGR students are.
- Research lens: what research they are attached to.
- Funding lens: who funds them and until when.
- Workforce lens: which students are also employees, sourced through the HR adapter.
- Statutory lens: fields required for external returns.

13.3 Dashboards

- Executive: totals, conversion rate, active researchers, funding coverage, students at risk, progression due, submissions, completion rate, average time to completion, funding expiry, supervisor load.
- Supervisor: caseload with stage, next milestone, funding state, and risk flag.
- Administrator: operational queues, applications awaiting assessment, offers awaiting acceptance, progression reviews due, funding expiring, examiner nominations awaiting approval, theses submitted, completion cases blocked.
- Student: journey timeline from registration to completion.

13.4 Exports

Large exports run as async jobs on the worker tier. The API returns a job id, the worker writes the file to the object store, and the client polls the job endpoint for a signed download link.

14. Frontend architecture (React)

The frontend is delivered by the frontend team. This section fixes the contract and a recommended structure so the two tiers fit together.

14.1 Contract

- The SPA authenticates through the platform auth endpoints and stores only short lived tokens in memory, with refresh handled silently.
- All data access is through `/api/v1`. No direct database or third party calls from the browser.
- The SPA reads `GET /api/v1/me` on load to shape navigation and hide actions the user cannot perform. The server still enforces every permission. Client hiding is convenience, not security.

14.2 Recommended structure

```
frontend/  
  src/  
    app/           # routing, providers, layout shells  
    features/      # one folder per capability, mirrors backend modules  
      persons/  
      recruitment/  
      admissions/  
      students/  
      supervision/  
      progression/  
      funding/  
      thesis/  
      completion/  
      dashboards/  
    shared/  
      api/         # generated client from OpenAPI  
      components/  # design system components  
      hooks/  
      auth/  
      types/       # types generated from OpenAPI  
  index.html  
  vite.config.ts
```

14.3 Recommended libraries

React with TypeScript, Vite for build, TanStack Query for server state, React Router for routing, a component library aligned to the brand palette, and a generated API client so types stay in step with the backend contract.

15. Non functional requirements and service levels

Area	Target
Availability	99.9 percent monthly for the API tier
Read latency	95th percentile under 300 ms for standard reads
Write latency	95th percentile under 600 ms for standard writes
Dashboard load	under 2 seconds using pre computed read models
Throughput	scales horizontally, sized to institution population
Recovery point objective	15 minutes via continuous backup
Recovery time objective	under 1 hour
Data retention	per institutional and statutory policy, configurable

16. Scalability and performance strategy

- Stateless API pods behind a load balancer. Scale out with a horizontal pod autoscaler on CPU and request latency.
 - Connection pooling through PgBouncer so many pods share a bounded set of database connections.
 - Read replicas absorb reporting and dashboard load. The application routes reads that tolerate slight lag to replicas.
 - Caching in Redis for reference data and for `GET /api/v1/me` role resolution, with short time to live and event based invalidation.
 - Async workers absorb spikes of notifications, milestone generation, and integration traffic, so the request path stays fast.
 - Append heavy tables partitioned by month to keep indexes small and vacuum cheap.
 - Materialized views and scheduled refresh keep expensive cross entity queries off the hot path.
 - Backpressure and rate limits at the gateway protect the platform from a misbehaving client or integration.
-

17. Security and compliance

- Transport encrypted with TLS. Data encrypted at rest at the storage layer.
 - Least privilege everywhere. Application database roles are scoped. Secrets come from a secret manager, never from the repository.
 - Input validated by Pydantic at the edge. Output schemas prevent accidental field leakage.
 - Authorization checked on every request. Fail closed.
 - Personal data handled under data protection rules. The person model supports data subject requests, and audit shows who accessed sensitive records.
 - Dependency and container scanning run in CI. Images are rebuilt on base image security updates.
 - Webhook endpoints verify signatures. Outbound integration uses mutual TLS or signed requests where the partner supports it.
-

18. Observability and operations

- Structured JSON logs with a request id that ties the SPA call, the API span, and any worker job together.
 - Distributed tracing through OpenTelemetry across API and workers.
 - Metrics in Prometheus, dashboards in Grafana, alerting on latency, error rate, queue depth, and integration failure rate.
 - Error tracking in Sentry with release tagging.
 - Health endpoints: `GET /health/live` for liveness and `GET /health/ready` for readiness including database and broker checks.
 - Runbooks in `docs/` for common operational tasks such as a stuck outbox, a replica lag spike, or a failed integration partner.
-

19. Deployment and infrastructure

19.1 Images and environments

- One backend image runs both the API and the workers, selected by the container command. This keeps build and dependency parity.
- Environments: dev, staging, production, each an overlay on the same Kubernetes base.

19.2 Kubernetes shape

```
api Deployment      # N replicas, HPA, readiness and liveness probes
worker Deployment   # N replicas for tasks
scheduler Deployment # single leader for periodic jobs
Service + Ingress   # routing, TLS, WAF
ConfigMap + Secret  # configuration and credentials
CronJob             # database maintenance, view refresh fallback
```

19.3 CI and CD

Pipeline stages:

1. Install and cache dependencies.
2. Lint and type check backend and frontend.
3. Run unit and integration tests against an ephemeral PostgreSQL.
4. Build and scan container images.
5. Run database migration check against a disposable database.
6. Deploy to staging, run end to end tests.
7. Promote to production with a rolling update and automatic rollback on failed health checks.

19.4 Data and backups

Continuous archiving of the database with point in time recovery. Backups tested by periodic restore into a scratch environment. Object store versioning on for document references.

20. Testing strategy

Level	Scope	Tooling
Unit	services and pure logic with repositories mocked	pytest
Integration	routers to database on an ephemeral PostgreSQL	pytest, httpx, testcontainers
Contract	OpenAPI schema checks so the frontend contract holds	schemathesis
End to end	key journeys across API and workers in staging	pytest or Playwright driven flows
Load	dashboard and list endpoints at target throughput	k6 or locust

Coverage focuses on lifecycle transitions, authorization and row scoping, funding change correctness, and the workflow engine, because those carry the most risk.

21. MVP scope and phasing

Aligned to the product blueprint.

Phase 1, MVP, PGR core lifecycle:

```
Person and identity -> Research opportunity -> PGR recruitment ->  
Application -> Assessment -> Offer -> Student registration ->  
Supervisor assignment -> Funding link -> Progression -> Thesis -> Completion
```

Deliver the person model, the recruitment pipeline for both routes, admission to student conversion on one preserved person id, the student record, one configurable progression milestone flow, a single funding arrangement, and a basic thesis to completion path. Ship the executive and administrator dashboards over the read models.

Phase 2:

Advanced progression workflows, funding changes with Finance notification, stipend integration, examiner management, notifications, student and supervisor portals, configurable workflows, and statutory reporting.

Phase 3:

PGR Enterprise 360 analytics, research and funding analytics, supervisor analytics, attrition and completion forecasting, funding and progression risk, and cross enterprise reporting.

22. Key design decisions

Decision	Choice	Reason
Service style	Modular monolith, extractable	Production capability without early microservice cost
Identity	One person, many relationships	Preserves the applicant to alumni thread
Lifecycle rules	Data driven definitions	Configurable per programme and institution
Funding	Separate arrangements over time	Supports movement between funders in one journey
Integration	Outbox plus adapters plus anti corruption layer	Reliable delivery, external systems stay authoritative
Reporting	Read replicas and materialized views	Fast dashboards without loading the write primary
Workflow	One reusable engine	Avoids hard coding every flow
Authorization	RBAC plus row scoping	Fine grained and enforced server side

Appendix A. Machine readable data dictionary

```
entities:
  person:
    pk: id
    fields: [external_person_ref, given_name, family_name, preferred_name,
             date_of_birth, email, nationality, created_at, updated_at]
    relations:
      - person_relationship: one_to_many
      - application: one_to_many
      - student: zero_or_one
  person_relationship:
    pk: id
    fields: [person_id, relationship_type, valid_from, valid_to, source_system]
    enums: { relationship_type: person_relationship_type }
  research_opportunity:
    pk: id
    fields: [title, research_area_id, department_id, principal_supervisor_id,
             research_award_id, funding_source_id, stipend_amount, currency,
             eligibility, start_date, expected_duration_months,
             positions_available, status]
    enums: { status: opportunity_status }
  application:
    pk: id
    fields: [person_id, route, research_opportunity_id, research_area_id,
             proposal_document_ref, current_stage, submitted_at]
    enums: { route: application_route, current_stage: candidate_stage }
  application_assessment:
    pk: id
    fields: [application_id, assessor_user_id, criteria, decision, rationale,
             assessed_at]
  offer:
    pk: id
    fields: [application_id, status, conditions, issued_at, responded_at,
             expires_at]
    enums: { status: offer_status }
  student:
    pk: id
    fields: [person_id, student_ref, programme_id, department_id,
             research_area_id, start_date, expected_end_date, study_mode, status]
    enums: { study_mode: study_mode, status: student_status }
  research_project:
    pk: id
    fields: [student_id, research_topic, research_award_id, research_group]
  supervisor_relationship:
    pk: id
    fields: [student_id, supervisor_person_id, role, status, valid_from, valid_to]
    enums: { role: supervisor_role, status: supervision_status }
  milestone_definition:
    pk: id
    fields: [programme_id, name, trigger, due_offset_days, required_documents,
             review_panel, assessment_criteria, possible_outcomes]
  milestone:
    pk: id
    fields: [student_id, milestone_definition_id, due_date, status]
    enums: { status: milestone_status }
  progression_review:
    pk: id
    fields: [milestone_id, student_submission_ref, supervisor_assessment,
             panel_decision, decided_by_user_id, decided_at, rationale]
    enums: { panel_decision: progression_outcome }
```

```
funding_arrangement:
  pk: id
  fields: [student_id, funding_type, funding_source_id, research_award_id,
           stipend_amount, currency, valid_from, valid_to, status]
  enums: { funding_type: funding_type, status: funding_status }
thesis:
  pk: id
  fields: [student_id, title, status, intention_to_submit_at, submitted_at,
           document_ref]
  enums: { status: thesis_status }
examiner_nomination:
  pk: id
  fields: [thesis_id, examiner_person_id, examiner_type, approved,
           approved_by_user_id]
examination:
  pk: id
  fields: [thesis_id, viva_date, outcome, decided_at]
  enums: { outcome: examination_outcome }
completion:
  pk: id
  fields: [student_id, status, requirements_met_at, award_confirmed_at,
           graduation_date]
  enums: { status: completion_status }
outbox_event:
  pk: id
  fields: [aggregate_type, aggregate_id, event_type, payload, created_at,
           dispatched_at]
audit_event:
  pk: id
  fields: [actor_user_id, action, aggregate_type, aggregate_id, before,
           after, request_id, created_at]
```

Appendix B. Standard error codes

validation_error	400	request failed schema or business validation
unauthenticated	401	missing or invalid token
permission_denied	403	authenticated but not allowed
not_found	404	resource does not exist or is out of scope
conflict	409	state or version conflict, includes stale ETag
idempotency_replay	200	returned cached result for a repeated key
rate_limited	429	too many requests
integration_error	502	an external system failed
workflow_error	422	an invalid lifecycle transition was requested
internal_error	500	unexpected failure, safe generic message

Appendix C. Environment variables

APP_ENV	dev staging production
APP_SECRET_KEY	signing key for tokens
DATABASE_URL	async PostgreSQL primary DSN
DATABASE_REPLICA_URL	read replica DSN, optional
REDIS_URL	cache and rate limit store
BROKER_URL	RabbitMQ or Redis broker for workers
OBJECT_STORE_ENDPOINT	S3 compatible endpoint
OBJECT_STORE_BUCKET	document bucket
OIDC_ISSUER	identity provider issuer URL
OIDC_CLIENT_ID	platform client id
OIDC_CLIENT_SECRET	platform client secret
ACCESS_TOKEN_TTL_SECONDS	short lived access token life
REFRESH_TOKEN_TTL_SECONDS	refresh token life
INTEGRATION_FINANCE_URL	finance adapter base URL
INTEGRATION_RESEARCH_URL	research adapter base URL
INTEGRATION_HR_URL	hr adapter base URL
SENTRY_DSN	error tracking, optional
OTEL_EXPORTER_ENDPOINT	trace collector endpoint
LOG_LEVEL	info debug warning

Appendix D. Glossary

PGR	Postgraduate research
Person 360	One person record across applicant, student, employee, alumni
Opportunity	A research position that can exist before a named student
Milestone	A configurable progression checkpoint for a student
Arrangement	A funding relationship for a period, one of several over time
Outbox	A table that guarantees reliable event delivery
Read model	A pre computed view built for fast reporting
Row scoping	Filtering rows a principal may see, applied server side
Anti corruption layer	An adapter that keeps external shapes out of the core

End of specification.